6. Level 1: System Context diagram

A System Context diagram can be a useful starting point for diagramming and documenting a software system, allowing you to step back and look at the big picture.

6.1 Intent

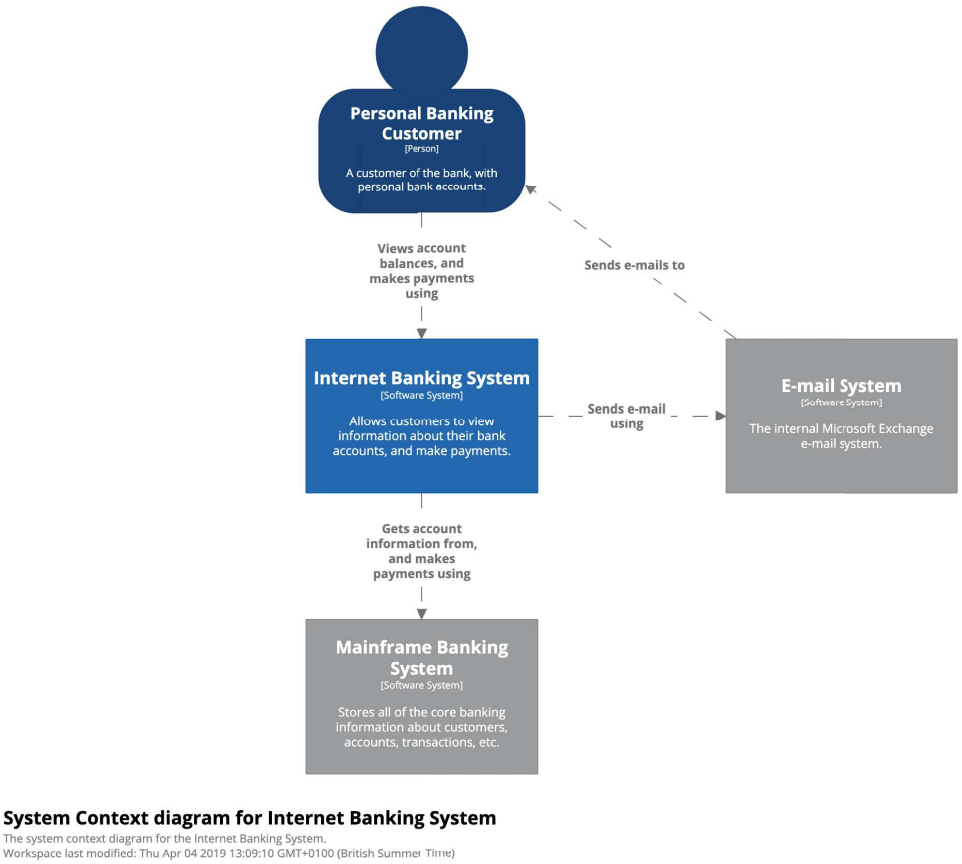
A System Context diagram helps you to answer the following questions.

1. What is the software system that we are building (or have built)?
2. Who is using it?
3. How does it fit in with the existing environment?

6.2 Structure

Draw a block diagram showing your software system as a box in the centre, surrounded by its users and the other software systems that it interacts with. Detail isn't important here as this is your zoomed-out view showing a big picture of the software system and the immediate world around it. The focus should be on people (actors, roles, personas, etc) and software systems rather than technologies, protocols and other low-level details. It's the sort of diagram that you could show to non-technical people.

Let's look at an example. This is a System Context diagram for a fictional Internet Banking System. It shows the people who use it, and the other software systems that the Internet Banking System has a relationship with.



A System Context diagram for the Internet Banking System

In summary, Personal Customers of the bank use the Internet Banking System to view information about their bank accounts, and to make payments. The Internet Banking System itself uses the bank's existing Mainframe Banking System to do this, and uses the bank's existing E-mail System to send e-mails to customers.

6.3 Elements

A System Context diagram usually includes two types of elements; people and software systems.

People

These are the people who use your software system. Whether you model them as individual people, users, roles, actors or personas is your choice. Typically I'll capture the following information about people:

- **Name**: The name of the person, user, role, actor or persona.
- **Description**: A short description of the person, their role, responsibilities, etc.

Software systems

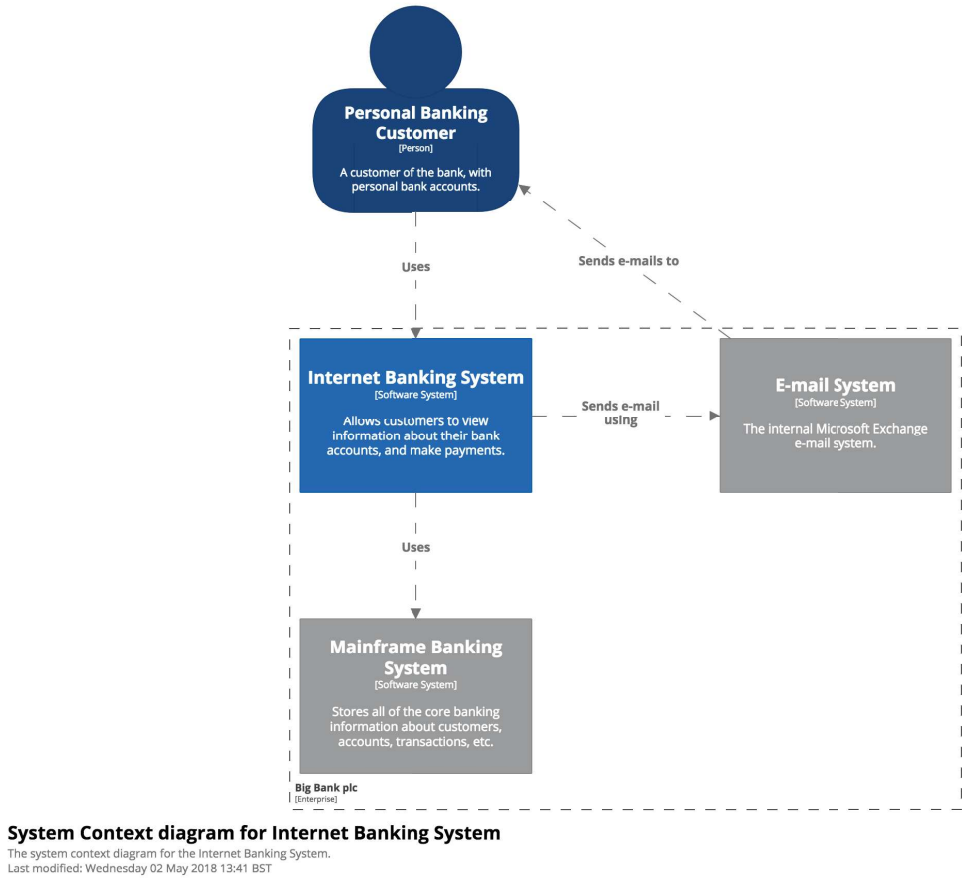
These are the other software systems that your software system interacts with. Typically these software systems sit outside the scope or boundary of your own software system, and you don't have responsibility or ownership of them. The Internet Banking example is very clear in this respect. As a team building the Internet Banking System, we don't own or have any responsibility over the bank's existing Mainframe Banking System, or the E-mail System. For this reason, they are included on the diagram to illustrate that they are dependencies of the Internet Banking System, and not something we are building ourselves.

Again, I'll capture the following information about each software system:

- **Name**: The name of the software system.
- **Description**: A short description of the software system, its responsibilities, etc.

Enterprise boundary

Optionally, I may want to capture some information about the location of people and/or software systems relative to my point of reference. If I'm building a software system inside an organisational/enterprise boundary, that software system may interact with people and software systems outside that boundary. In this slightly modified example, the dashed line represents the boundary of the bank, and is used to illustrate what's inside vs what's outside of the bank.



A System Context diagram for the Internet Banking System, additionally showing the enterprise boundary

6.4 Interactions

Try to annotate every interaction between elements on the diagram with some information about the purpose of that interaction. This avoids creating a diagram where a collection of boxes are somehow connected via a set of ambiguous lines. Again, try to keep this relatively high-level, and don't feel that you need to include lots of technical details (e.g. protocols, data formats, etc).

6.5 Motivation

You might ask what the point of such a simple diagram is. Here's why it's useful:

- It makes the context and scope of the software system explicit so that there are no assumptions.

- It shows what is being added (from a high-level) to an existing environment.
- It's a high-level diagram that technical and non-technical people can use as a starting point for discussions.
- It provides a starting point for identifying who you potentially need to go and talk to as far as understanding inter-system interfaces is concerned.

A System Context diagram doesn't show much detail but it does help to set the scene, and is a starting point for other diagrams. I will often draw this diagram during a requirements gathering workshop, to ensure that everybody understands the scope of what we've been tasked to build. It can be a great requirements gathering and analysis tool.

6.6 Audience

Technical and non-technical people, inside and outside of the immediate software development team.

6.7 Required or optional?

All software systems should have a System Context diagram.